

[C] “Yeah, Right...” Sarcasm Classification on Social Media

Arvid Kolstad
(Dated: June 5, 2026)

Sarcasm is a linguistically subtle phenomenon where the absence of prosodic cues makes automated detection particularly challenging. This project investigates three machine learning models, Logistic Regression, Bidirectional LSTM with multi-head attention, and BERTweet, trained to classify sarcastic comments drawn from a balanced Reddit dataset of one million entries. Logistic Regression serves as a baseline and performs only marginally above random chance, BiLSTM achieves a substantial improvement, and BERTweet attains the strongest performance, though it remains below state-of-the-art benchmarks. The results suggest that dataset noise from self-labeling and the absence of conversational context pose challenges that model capacity alone cannot fully overcome.

Project Topic: Sarcasm Detection in Social Media

Teaching Assistant: Antonio Ciarlo

I. INTRODUCTION

Human communication is a rich and multifaceted phenomenon, shaped by tone, context, cultural background, and shared understanding between speakers. Among its many complexities, sarcasm stands out as a particularly challenging form of expression, one that relies heavily on non-verbal cues such as intonation, facial expression, and vocal emphasis to convey a meaning that is often the direct opposite of what is literally stated. While this poses little difficulty in face-to-face conversation, the written word strips away these contextual signals, leaving the reader to infer intent from text alone.

This challenge is amplified dramatically in the context of social media, where vast volumes of informal, unstructured text are produced daily across platforms dedicated to virtually every topic imaginable. On these platforms, sarcasm is not only common but often central to the culture of communication, used for humor, criticism, and commentary alike. The absence of prosodic cues in this medium means that sarcastic remarks are frequently misread, even by human readers familiar with the context.

For automated systems tasked with making sense of this content, whether for sentiment analysis, content moderation, or opinion mining, the accurate detection of sarcasm represents a significant open problem. A model that fails to recognize sarcasm will systematically misinterpret the sentiment and intent behind a large portion of online discourse, undermining the reliability of any downstream analysis.

This report investigates the capacity of machine learning models to address this problem by classifying sarcastic statements drawn from social media text. By evaluating and comparing different modeling approaches, this work aims to shed light on how well computational systems can grasp one of the more elusive dimensions of human language.

II. OVERVIEW

In this section a brief summary of the models used in this report can be found. To every model, there is a short discussion about its advantages and weak point and an argument why the model was used.

Method 1 - Logistic Regression.

This model is rather simple and uses only a linear layer to classify the sentence from the vectorization. For this it uses the pre-trained model FastText [1] which is a more complex version of the famous Word2Vector. It can handle the problem of Out-Of-Vocabulary words by breaking up longer words into smaller parts that it does understand. This is great for social media text because of its higher frequency of typos and slang. Due to the models simplicity, it does not understand words connection to other words and therefore this is a good model to use as a benchmark.

Method 2 BiLSTM with multi-head attention.

This model uses a recurrent network that takes each word vector, both from the start of the sentence, and the end of the sentence, and feed these trough with the next word as input. After this the hidden states of the network gets fed into an attention network which learns how different words connects other. This is a good in between the Logistic Regression and the BERT-model and will hopefully display the importance of connections between words when it comes to classifying sarcasm.

Method 3 BERT.

This model leans heavily on the pre-trained BERTweet model [2], and leverages its understanding of language to try to predict sarcasm. The BERTweet is a transformer encoder that first has been trained on a big dataset of english, and then have been fine tuned on 850 million tweets. This makes it good at understanding internet language and slang, which makes it perfect for this classification problem. It's without question the biggest model in this project and has over 100 million parameters, and has the best chances of scoring the highest.

TABLE I: **Overview of ML models used in this project.** Table where the different models used in the project is presented, their features explained and arguments for their fit to the project.

Method	Use case scenario	Features	Suitable for the project?
Logistic regression	Classification Model	Fast and simple model that is easy to interpret and train. Looks at the importance of each word in the sentence to classify it. Probably too simple for classifying harder sentences.	Well studied method that is used for getting a base line performance on the data set.
BiLSTM with multi-head attention	Classification Model	Can capture word order and connect words over larger distances in sentences which is important for sarcasm. The attention mechanism can learn what words refers to each other and therefore understand the sentence better.	Medium size model that had great performance for its time.
BERTweet	Classification Model	An already pre-trained language model that can be used to classify sarcasm by setting a linear layer after it and fine-tuning the whole model on the specific data set. The model is quite large and therefore it can be computationally intensive to train.	A large model that has a good understanding of our language, which makes it perfect for using in this classification problem.

III. METHOD

B. FastText

In this section we will first introduce the dataset and then we will go through the different models architecture and then explain the training pipeline. All the models are built in python by using the library PyTorch.

A. Dataset

The dataset that has been used in this project is the following [3]. It's a dataset that was retrieved from the website Kaggle.com, and has been created by scraping the the social platform Reddit.com for comments. On the website you can put different labels on your comments to signal what the comment is about. One of these labels stands for that the comment has some sarcastic remarks in it, and this is what is used for classifying the dataset. The total number of comments is exactly 1 million and the dataset is perfectly balanced so 500K is labeled as normal and the other half is labeled as sarcastic. The dataset also contained the text that the comment answered to, what the authors name was, the name of the specific chat forum that the comment was written in and much more but the only data that was used in this project was the original comment and its sarcasm label.

For both this model and the BiLSTM, the word embedding tool FastText is used [1]. It is a pre trained model that takes a word and transforms it into a 300 dimensional vector, and therefore makes it easier interpretable to a computer. This model was picked for the project for its ability to handle Out-Of-Vocabulary words. If it stumbles upon a word that it doesn't recognize it tries to break down the word in smaller words that could make the word make sense. This is great for handling slang and spelling error that might not be included in the FastText set of known words. This model only handles word so special characters are removed before converting the word to a sentence. For sarcasm in text, these provide valuable markers so to not loose this information, 4 sarcasm pointer where extracted from each sentence. The pointers where words with where all letters where capitalized, and the sequences "!!!", "?!" or "!?", and "...". Other papers like [4] and [5] use more sarcasm pointers with higher complexity than what's used in this project, but with regards to time scope of the project it was decided that these four pointers where enough.

C. Logistic Regression

This model serves as the simplest baseline and is expected to face challenges in capturing the nuances of sarcasm. Because the comments vary in length, and the logistic regression model requires a fixed-size input, a mean sentence vector was computed by averaging the FastText embeddings of all words within a given comment. A consequence of this pooling approach is that the model can only capture the global semantic theme of the comment, while all sequential relationships and syntactic dependencies between words are lost. This sentence vector is subsequently concatenated with the sarcasm pointers and fed through a linear layer to produce a single scalar value. Finally, classification is performed using a sigmoid activation function applied against a specified threshold. A comprehensive flowchart of this baseline architecture is illustrated in Figure 1.

D. BiLSTM with Multi-Head Attention

This architecture is highly inspired by the work of [5], with the primary modification being the inclusion of fewer sarcasm pointers. In contrast to the Logistic Regression baseline, words are first mapped to vectors using FastText to construct a sequence matrix, preserv-

ing the individual word vectors rather than averaging them.

This sequence is then fed into a Bidirectional Long Short-Term Memory (BiLSTM) network. The bidirectional nature ensures that the network processes the text both forward (from the first to the last word) and backward (from the last to the first word), generating two distinct hidden states at each time step. Consequently, the hidden state representation of a specific word is influenced by both its preceding and succeeding context.

The complete set of hidden state vectors from both directions is subsequently passed to an attention layer to compute self-attention weights. This is achieved by passing the hidden states through two linear layers interspersed with a hyperbolic tangent (tanh) activation function, followed by a softmax function to normalize the weights. Lastly, these weights are multiplied by the original hidden vectors, and the resulting matrix is flattened and concatenated with the sarcasm pointers. A final linear layer is then utilized for the final classification. The corresponding flowchart for this model is detailed in Figure 1.

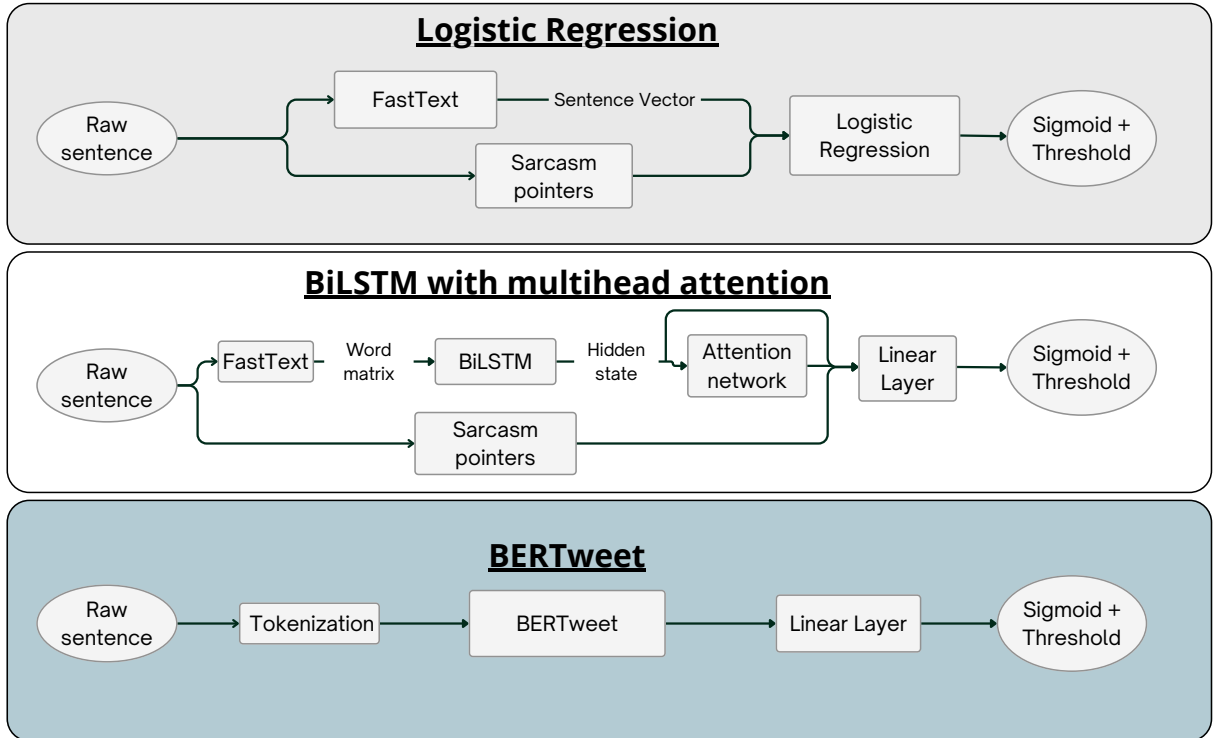


FIG. 1: **Flowchart of the models used in the project:** Architectural comparison of three sarcasm detection models. Top: a Logistic Regression baseline combining FastText sentence vectors and sarcasm pointers. Middle: a BiLSTM with multihead attention, using FastText word embeddings and sarcasm pointers as input. Bottom: a BERTweet transformer model with tokenization and a linear layer.

E. BERTweet

To leverage a large-scale pre-trained language model, a BERTweet architecture [2] was implemented and fine-tuned for the sarcasm classification task. Unlike the previous models, additional explicit sarcasm pointers were omitted for this configuration; the specialized BERTweet tokenizer natively retains and processes emojis, hashtags, and special characters, allowing the model to encode these stylistic cues implicitly.

Instead of generating static embeddings or processing sequences through recurrent layers, the input tokens are passed through the transformer blocks to capture deep, bidirectional contextual representations. For the final classification task, the embedding of the special classification token ([CLS]), which aggregates the semantic representation of the entire sequence, is extracted from the final hidden layer. This token embedding is then fed directly into a final linear layer to project the representation to a single logit, which is subsequently classified using a sigmoid function and a threshold. The architectural overview of this fine-tuned model is presented in Figure 1.

F. Model and Training configuration

The dataset was divided into three different sets: Training (50%), Validation (25%) and Test (25%) where the class balance where kept. The training set was used to train the models parameters, the validation set was used for stopping training and also to set the final threshold by iterating over each threshold and picking the best one, and the test set was used to get the results. The total parameter in the Logistic Regression is 304 parameters, while the BiLSTM contains 474 629 parameters and last but not least the BERTweet with 134 900 737 parameters.

IV. RESULTS AND DISCUSSION

In this section the results are presented and discussed. Table II shows the accuracy and F1-score for each model. Logistic Regression performs only marginally above random chance, which is expected given the model’s limited capacity for capturing contextual information. BiLSTM with multi-head attention achieves a substantial improvement, performing comparably to the model proposed by [5]. BERTweet achieves the highest performance of the three models; however, it falls notably short of state-of-the-art results reported in [6]. This gap may be attributed to several factors, including computational constraints and dataset quality. The dataset was constructed by scraping Reddit posts where users self-labeled their comments as sarcastic, which introduces noise — there

Method	Accuracy	F1-Score	Threshold
Logistic Regression	0.580	0.680	0.380
BiLSTM	0.702	0.732	0.320
BERTweet	0.779	0.792	0.330

TABLE II: Results for the three models. Logistic Regression is just better than guessing random. BiLSTM makes a big jump up on both accuracy and F1-Score, but BERTweet is still far better. We can also see that for all the models the thresholds are all low.

is no guarantee that sarcastic comments without such labels were excluded from the non-sarcastic class.

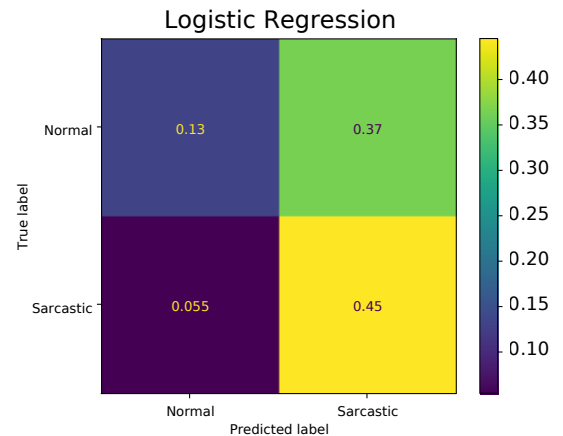


FIG. 2: **Confusion Matrix for Logistic Regression** In the figure we can see how the model almost always guesses sarcastic, which signals that this model architecture can not understand sarcasm.

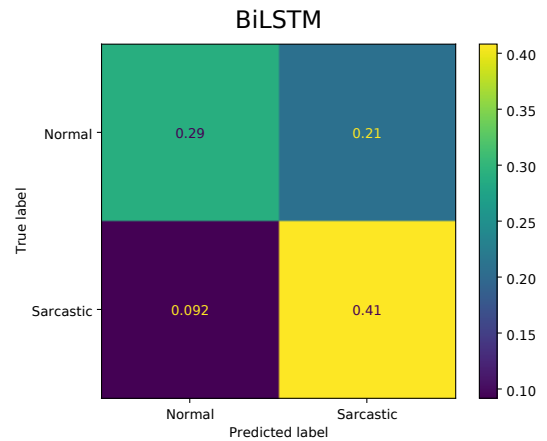


FIG. 3: **Confusion Matrix for BiLSTM** The BiLSTM performs better and the classifications are more balanced, but still overestimates the sarcastic classifications.

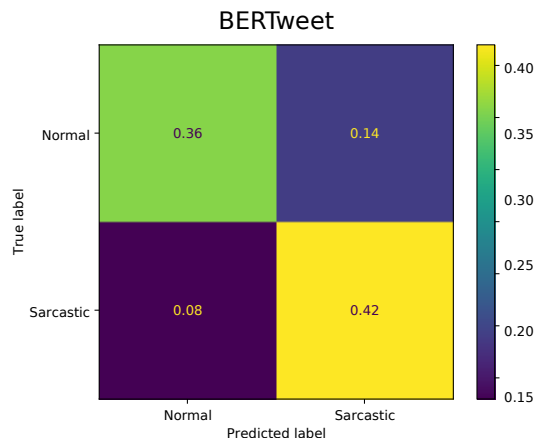


FIG. 4: **Confusion Matrix for BERTweet** The best performing model and which also has the best balance between sarcastic and normal classifications.

The confusion matrices reveal the classification strategy adopted by each model. As shown in figure 2, Logistic Regression defaults to predicting sarcasm for the majority of inputs, reserving the non-sarcastic class only for high-confidence cases. Figure 3 demonstrates that BiLSTM produces considerably more balanced predictions, suggesting that the self-attention mechanism meaningfully improves the model’s ability to detect sarcastic language. Figure 4 shows the most balanced classification of the three, with false positive and false negative rates approaching parity. Notably, all models converge on a similar optimal threshold well below 0.5, despite being trained with a threshold of 0.5 that was subsequently adjusted using validation data. This consistent pattern suggests that sarcastic signals in the data are weak, and that models tend to assign low-confidence probabilities even to genuinely sarcastic samples. One possible explanation is that the self-labeling nature of the dataset introduces noise into the sarcastic class, causing models to assign cautiously low probabilities across the board. A reduced threshold is then necessary to recover meaningful

recall, consistent with the general bias toward sarcastic predictions seen across all confusion matrices.

V. CONCLUSIONS AND OUTLOOK

Logistic Regression demonstrates limited capacity for reliable sarcasm detection, while BiLSTM and BERTweet both achieve considerably stronger performance given the constraints of the dataset. A potential avenue for improvement would be to incorporate contextual information, such as the parent post to which a comment responds, as sarcasm is often dependent on the broader conversational context. This is particularly relevant for short social media comments, where the absence of such context makes classification challenging even for human annotators.

VI. CONTRIBUTIONS

Kolstad, A. conceived and implemented the code for the project. Kolstad, A. created the poster for the presentation of the project. Kolstad, A. wrote the report for the project.

VII. CONFLICT OF INTEREST

The author declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

VIII. DATA AND CODE AVAILABILITY

The code can be found at: <https://github.com/ArvidKolstad/Advanced-Neural-Network> and the dataset can be found at <https://www.kaggle.com/datasets/danofer/sarcasm/data>

-
- [1] E. Grave, P. Bojanowski, P. Gupta, A. Joulin, and T. Mikolov, Learning word vectors for 157 languages, *Proceedings of the International Conference on Language Resources and Evaluation (LREC 2018)*, (2018), reference for the pretrained FastText model.
 - [2] D. Q. Nguyen, T. Vu, and A. T. Nguyen, BERTweet: A pre-trained language model for English Tweets, *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, , 9 (2020), for the pretrained BERTweet model.
 - [3] M. Khodak, N. Saunshi, and K. Vodrahalli, A large self-annotated corpus for sarcasm (2017), dataset that was used.
 - [4] M. S. Razali, A. A. Halin, L. Ye, S. Doraisamy, and N. M. Norowi, Sarcasm detection using deep learning with contextual features, *IEEE Access* **9**, 68609 (2021), uses the FastText, instead of the more common Word2Vec, where

words get broken up into smaller bits, which handles spelling errors better, which is a common problem with social media texts.

- [5] A. Kumar, V. Narapareddy, V. Aditya Srikanth, A. Malapati, and L. Neti, Sarcasm detection using multi-head attention based bidirectional lstm doi.org/10.1109/ACCESS.2019.2963630 (2020), a paper on the use of LSTM with Multi-Head Attention which is used in in paper.
- [6] E. Sharma, N. Gondhi, Chaahat, M. Murugappan, G. Naik, and M. L., Towards effective sarcasm detection in social media: A review of artificial intelligence-based automated approaches <https://doi.org/10.1111/exsy.70203> (2026), a great summary on the research of trying to classify Sarcasm on Social Media with artificial intelligence.